Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 10

CI/CD in Practice:
Pipelines, GitHub Actions & Deployment Strategies

Team Standup Meetings

Last Week

Software Testing

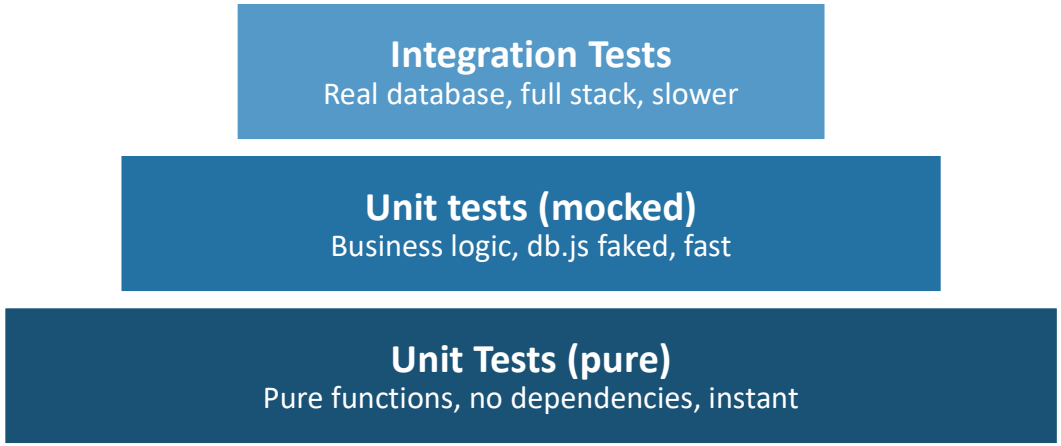
The Friday Afternoon Push

You push code at 4:45 PM on a Friday.
How do you know it didn't break anything?

❌ Hope for the best!
❌ Run it manually
❌ Ask a teammate
✅ CI catches it

CI/CD is the automated answer to the question.
Continuous Integration — merge often, catch breaks early
Continuous Delivery — ship with confidence, not ceremony

The Testing Pyramid (Automated)



Today’s demo covers all three layers – pure logic, mocked DB, and real database integration

Containers: Just Enough to Understand CI Runners

👨‍💻 Your laptop – Node 18, macOS, Your env vars, Your npm cache

👩‍💻 Teammate's laptop – Node 20, Windows, Different env vars, nothing cached

📦 **A container** – Node 20, Ubuntu, Exactly the dependencies in package.json. Every time.

Image = The recipe
Container = A running version of the image
--rm = Throws it away after

$docker run –rm node:20 node –e "console.log(`Node ` + process.version)"
Node v20.x.x

Pipeline Structure: Reading the YAML

“Yet another markdown language”
“YAML ain’t markdown language?”

```
# CI Workflow — Week 10 Demo
#
# THREE jobs, each explaining a different layer:
#
#   lint        — static analysis, no runtime needed
#   unit-tests     — pure logic, db.js mocked, no database
#   integration    — real Postgres spun up as a service container
#
name: CI
on:
     push:
          branches: ["**"]
     pull_request:
          branches: [main]
jobs:
     # ── Job 1: Lint
     lint:
          name: Lint
          runs-on: ubuntu-latest
          steps:
               - uses: actions/checkout@v4
               - uses: actions/setup-node@v4
                 with:
                    node-version: "20“
                    cache: "npm“
               - run: npm ci
               - run: npm run lint
```

Pipeline Structure: Reading the YAML (continued)

```
# ── Job 2: Unit Tests
# db.js is mocked – no database needed at all.
unit-tests:
     name: Unit Tests
     runs-on: ubuntu-latest
     needs: lint
     steps:
          - uses: actions/checkout@v4

          - uses: actions/setup-node@v4
            with:
               node-version: "20“
               cache: "npm”

          - run: npm ci

          - name: Run unit tests with coverage
          - run: npm run test:unit

          - name: Upload coverage report
            uses: actions/upload-artifact@v4
            with:
               name: coverage-report
               path: coverage/
               retention-days: 7
```

Pipeline Structure: Reading the YAML (continued)

```
    # ── Job 3: Integration Tests
    # GitHub spins up a real Postgres container alongside this job.
    # The tests connect to it just like they would a prod database.
    integration:
        name: Integration Tests
        runs-on: ubuntu-latest
        needs: unit-tests
        services:
            postgres:
                image: postgres:15
                env:
                    POSTGRES_USER: postgres
                    POSTGRES_PASSWORD: postgres
                    POSTGRES_DB: ci_demo
                ports:
                    5432:5432
                options: >-
                    --health-cmd pg_isready
                    --health-interval 10s
                    --health-timeout 5s
                    --health-retries 5
        env:
            PGHOST: localhost
            PGPORT: 5432
            PGDATABASE: ci_demo
            PGUSER: postgres
            PGPASSWORD: postgres
```

Pipeline Structure: Reading the YAML (continued)

```
steps:
     -uses: actions/checkout@v4

     -uses: actions/setup-node@v4
      with:
          node-version: “20”
          cache: “npm”

     -run: npm ci

     -name: Run integration tests
      run: npm run test:integration
```

Pipeline Structure: Reading the YAML (continued)

```
# ── Job 4: Security Audit

audit:
     name: Security Audit
     runs-on: ubuntu-latest
     needs: integration
     steps:
          - uses: actions/checkout@v4

          - uses: actions/setup-node@v4
            with:
               node-version: "20”
               cache: "npm”
          - run: npm ci
          - run: npm audit --audit-level=moderate
```

Pipeline Structure: Reading the YAML - Reference

on:
The trigger – when does the pipeline wake up?

jobs:
Units of work. Each job gets its own fresh container.

runs-on:
Which container GitHub spins up, e.g. ubuntu-latest

services:
Sidecar containers (databases, etc.) started alongside the job.

needs:
Job dependency. Unit-tests only run if lint passes.

uses:
Pre-built action (someone else wrote this step).

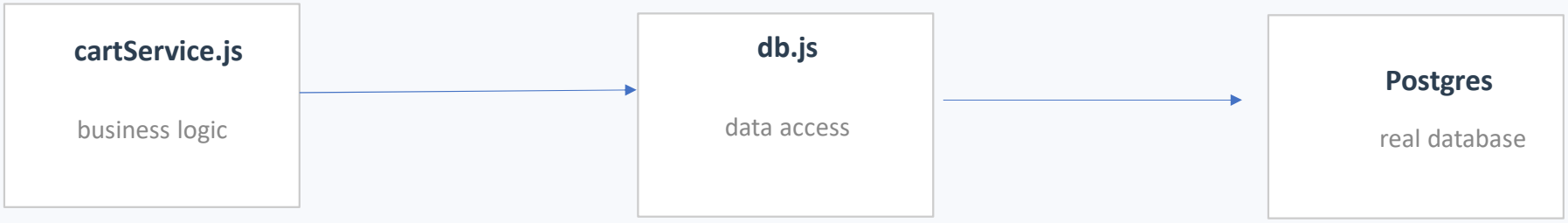
run:
A plain shell command. Anything you'd type in terminal.

env:
Environment variables – how the job finds the database.

Unit Tests with Mocking: Core Concept

~5 min

The architecture that makes mocking possible:




```
//In cartService.test.js:
jest.mock("../../..src/db");              //replaces db.js with fakes

db.getCart.mockResolvedValue(null)        //we control what "DB" returns
db.saveCart.mockResolvedValue();          //pretend save always succeeds

//Now test the logic:
const result = await addItem("user-1", { name: "Widget", price: 10, quantity: 2});
expect(result).toEqual([{name: "Widget", price: 10, quantity: 2});
```

No database or network needed. We decide what the database returns.

Deployment Strategies

- **Rolling Deploy**

  One environment. Replace instances one at a time, monitoring metrics. Old and new briefly coexist.
- **Blue/Green**

  Two identical environments. Flip traffic from blue to green. Rollback means flip back.
- **Canary Release**

  Send 5% of traffic to new version. Watch metrics. Gradually increase (or roll back).

Test Plan Assignment

Four Deliverables

1. **Unit Tests**
   ~~Every method, branch line – 100% coverage~~
   Use mocks for anything that touches a database or API.
   Use Jest/pytest/JUnit
2. **Integration Tests**
   Full stack against real infrastructure. No mocks. Only applies if your project uses a database.
3. **User Acceptance Tests**
   Written, step-by-step script a non-technical user can follow. One per functional requirement. Include expected outcomes.
4. **GitHub Actions Automation**
   Workflow running unit tests + coverage on every push. If you have integration tests, add a second job with a service container. Start from the demo repo's ci.yml.

Wiring Your Project's CI

Language	Setup Action	Install	Test Command
Python	actions/setup-python@v5	pip install -r requirements.txt	pytest --cov=src/ --cov-report=xml
Java	actions/setup-java@v4	(Maven/Gradle handles deps)	mvn test or gradle test
JavaScript	actions/setup-node@v4	npm ci	npm test
C#	actions/setup-dotnet@v4	dotnet restore	dotnet test --collect:"XPlat Code Coverage"

Start from our demo ci.yml — swap the setup action + test command for your language.

If your project uses a database: add a services: block to the integration job – just like we did with Postgres.

Questions to answer for your project:

- What test runner are you using?
- Do you have a database? (if yes — integration tests!)
- What triggers a deploy?

You should now be able to:

- Read and write a GitHub Actions workflow YAML
- Explain why unit tests use mocks, what they can and can’t catch
- Write unit tests that mock a database dependency
- Write integration tests against a real service container
- Interpret pipeline failure logs at each layer
- Wire your project’s tests into an automated CI pipeline

Next Week:

Maintenance, Refactoring & Technical Debt

Assignment: Weekly Standup Reflection

https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection

(Individual Assignment)
Due Feb 26 (and every week thereafter)

Assignment: The Overdose

https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose

(Individual Assignment)
Due Apr 23

Assignment: Software Test Plan

https://rrostock1.github.io/Ursinus-CS375/Project/TestPlan

(Group Assignment)

Due Apr 23